Manufacturing allocation intelligence system with dual-circuit economics

---

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
MANUFACTURING ALLOCATION INTELLIGENCE SYSTEM (SAAS CORE)
Cycle-aware raw material allocation, demand forecasting, and policy signals
Aligned with dual-circuit economics (FDR) and counter-cyclical principles

Standard library only: ready to copy/paste and run.
"""

import math
import json
import random
from datetime import datetime, timedelta
from urllib.request import urlopen

# ------------------------------
# Utility functions (standard lib)
# ------------------------------

def clamp(x, lo, hi):
    return max(lo, min(hi, x))

def safe_div(a, b, default=0.0):
    return a / b if b else default

def moving_average(series, window):
    if window <= 0 or not series:
        return 0.0
```

```
    window = min(window, len(series))
    return sum(series[-window:]) / window

def exponential_smoothing(series, alpha=0.4):
    """Simple ESO forecast; alpha ∈ (0,1]"""
    if not series:
        return 0.0
    s = series[0]
    for x in series[1:]:
        s = alpha * x + (1 - alpha) * s
    return s

# ------------------------------
# Dual-circuit economics (FDR)
# ------------------------------

class DualCircuitEconomics:
    """
    Financial Decoupling Ratio (FDR) proxy and regime detector.
    No external dependencies. Falls back to realistic defaults that reflect asset-led booms
and busts.
    """

    def __init__(self):
        self.data = {}
        self.fdr = 1.0
        self.regime = "HEALTHY_EXPANSION"  # default

    def fetch(self):
        try:
            # If your API exists, use it. Otherwise, we use principled defaults.
            with urlopen('https://api.factoryofthefuture.ai/economic-indicators') as response:
                data = json.loads(response.read().decode())
        except Exception:
            # Fallback realistic snapshot (proxy metrics)
            data = {
                "manufacturing_pmi": 52.5,
```

```
            "core_pce": 0.032,          # 3.2%
            "ci_loans_growth": 0.03,     # 3% YoY growth
            "margin_debt_growth": 0.08,  # 8% YoY growth (risk-on)
            "asset_price_index": 1.12,   # 12% index uplift
            "last_updated": datetime.now().isoformat(),
            "source": "Dual-Circuit Model (fallback)"
        }
    self.data = data
    self._compute_fdr()
    self._detect_regime()
    return self.data

  def _compute_fdr(self):
    # FDR ≈ (Growth in asset-backed credit × Aggregate asset price index) / (Growth in
C&I loans × Core PCE)
    asset_growth = self.data.get("margin_debt_growth", 0.06)
    asset_index  = self.data.get("asset_price_index", 1.08)
    ci_growth    = self.data.get("ci_loans_growth", 0.02)
    core_pce     = self.data.get("core_pce", 0.03)
    denom = ci_growth * core_pce if ci_growth * core_pce > 0 else 0.0001
    self.fdr = clamp((asset_growth * asset_index) / denom, 0.2, 5.0)

  def _detect_regime(self):
    # Regimes aligned with your doctrine
    if self.fdr > 2.0:
        self.regime = "IMBALANCED_EXCESS"          # bubble risk
    elif self.fdr > 1.5:
        self.regime = "ASSET_LED_GROWTH"           # yellow alert
    elif self.fdr < 0.8:
        self.regime = "REAL_ECONOMY_LEAD"          # bust/repair
    else:
        self.regime = "HEALTHY_EXPANSION"

  def signals(self):
    s = []
    if self.regime == "IMBALANCED_EXCESS":
        s += [
```

```
            {"signal": "REDUCE_INVENTORY", "intensity": 0.8},
            {"signal": "TIGHTEN_CREDIT_TERMS", "intensity": 0.7},
            {"signal": "DEFER_EXPANSION_CAPEX", "intensity": 0.9},
            {"signal": "REFINANCE_LONG_FIXED", "intensity": 0.9}
        ]
    elif self.regime == "ASSET_LED_GROWTH":
        s += [
            {"signal": "SLOW_INVENTORY_BUILD", "intensity": 0.6},
            {"signal": "EXTEND_PAYABLES", "intensity": 0.6},
            {"signal": "LOCK_RATES", "intensity": 0.7}
        ]
    elif self.regime == "REAL_ECONOMY_LEAD":
        s += [
            {"signal": "COUNTER_CYCLICAL_BUY", "intensity": 0.9},
            {"signal": "EXPAND_CAPACITY_SELECTIVE", "intensity": 0.7},
            {"signal": "SUPPLY_CHAIN_FINANCE", "intensity": 0.8}
        ]
    else:
        s += [
            {"signal": "OPTIMIZE_TURNOVER", "intensity": 0.5},
            {"signal": "SELECTIVE_CAPEX", "intensity": 0.5}
        ]
    return s

# ------------------------------
# Demand forecaster (standard lib)
# ------------------------------

class DemandForecaster:
    """
    Multi-horizon demand forecast per SKU using:
    - Baseline: exponential smoothing of historical shipments
    - Cycle-aware adjustment: FDR regime factor
    """

    def __init__(self, history_by_sku):
        # history_by_sku = {sku: [monthly_units ...]}
```

```
        self.history_by_sku = history_by_sku or {}

    def forecast(self, sku, months_ahead=3, regime="HEALTHY_EXPANSION"):
        history = self.history_by_sku.get(sku, [])
        base = exponential_smoothing(history, alpha=0.45)
        # Regime multipliers: conservative in bubbles, assertive in busts
        regime_factor = {
            "IMBALANCED_EXCESS": 0.92,
            "ASSET_LED_GROWTH":  0.96,
            "HEALTHY_EXPANSION": 1.00,
            "REAL_ECONOMY_LEAD": 1.05
        }.get(regime, 1.0)
        # Simple seasonality proxy via last 12-month average, if available
        seasonality = moving_average(history, 12)
        season_factor = 1.0
        if seasonality > 0:
            season_factor = clamp(base / seasonality, 0.85, 1.15)

        # Project forward with damping
        forecasts = []
        level = base * regime_factor * season_factor
        for h in range(1, months_ahead + 1):
            damp = 1.0 - 0.06 * (h - 1)
            forecasts.append(max(0.0, level * damp))
        return forecasts

# ------------------------------
# Raw material allocation engine
# ------------------------------

class AllocationEngine:
    """
    Allocate raw materials across SKUs given:
    - Forecast demand
    - BOM consumption rates per unit
    - Inventory and procurement constraints
    - Cycle-aware policy signals (from FDR)
```

```
"""

def __init__(self, bom, on_hand, inbound, budget, supplier_terms):
    """
    bom: {sku: {material: units_per_sku}}
    on_hand: {material: units}
    inbound: {material: units arriving within the horizon}
    budget: float (procurement budget $)
    supplier_terms: {material: {"unit_cost": $, "lead_time_days": int}}
    """
    self.bom = bom
    self.on_hand = on_hand
    self.inbound = inbound
    self.budget = budget
    self.supplier_terms = supplier_terms

def _material_need_for_sku(self, sku, planned_units):
    need = {}
    for m, per_unit in self.bom.get(sku, {}).items():
        need[m] = need.get(m, 0.0) + per_unit * planned_units
    return need

def _available_material(self, material):
    return self.on_hand.get(material, 0.0) + self.inbound.get(material, 0.0)

def plan(self, forecast_by_sku, regime_signals, priority_by_sku):
    """
    forecast_by_sku: {sku: units to produce within horizon}
    regime_signals: list of dicts from DualCircuitEconomics.signals()
    priority_by_sku: {sku: weight} higher → more strategic

    Returns dict with allocation plan, procurement plan, and KPIs.
    """
    # Interpret regime signals into policy knobs
    knobs = {
        "inventory_buffer_factor": 1.0,  # raise in bust, cut in bubble
        "tighten_credit": False,
```

```
    "extend_payables": False,
    "capex_gate": False
}
for s in regime_signals:
    if s["signal"] in ("REDUCE_INVENTORY", "SLOW_INVENTORY_BUILD"):
        knobs["inventory_buffer_factor"] *= (1.0 - 0.15 * s["intensity"])
    if s["signal"] in ("TIGHTEN_CREDIT_TERMS",):
        knobs["tighten_credit"] = True
    if s["signal"] in ("EXTEND_PAYABLES",):
        knobs["extend_payables"] = True
    if s["signal"] in ("DEFER_EXPANSION_CAPEX",):
        knobs["capex_gate"] = True

# Normalize priorities
total_w = sum(priority_by_sku.values()) or 1.0
norm_priority = {sku: w / total_w for sku, w in priority_by_sku.items()}

# Initial production targets (policy-aware buffer)
production_targets = {}
for sku, units in forecast_by_sku.items():
    buffer = 0.10 * knobs["inventory_buffer_factor"]  # 10% baseline
    production_targets[sku] = max(0.0, units * (1.0 + buffer))

# Compute material needs across all SKUs
aggregate_needs = {}
sku_material_needs = {}
for sku, units in production_targets.items():
    need = self._material_need_for_sku(sku, units)
    sku_material_needs[sku] = need
    for m, q in need.items():
        aggregate_needs[m] = aggregate_needs.get(m, 0.0) + q

# Available materials
available = {m: self._available_material(m) for m in aggregate_needs}

# Allocation heuristic: priority-weighted fairness
sku_allocation = {sku: 0.0 for sku in production_targets}
```

```
        material_remaining = available.copy()

        # Greedy rounds to satisfy needs respecting material constraints
        for sku, units in sorted(production_targets.items(), key=lambda kv:
-norm_priority.get(kv[0], 0)):
            # Max feasible units limited by each material
            caps = []
            for m, per_unit in self.bom.get(sku, {}).items():
                if per_unit > 0:
                    caps.append(material_remaining.get(m, 0.0) / per_unit)
            feasible_units = min([units] + caps) if caps else 0.0
            feasible_units = max(0.0, feasible_units)
            # Allocate
            sku_allocation[sku] = feasible_units
            for m, per_unit in self.bom.get(sku, {}).items():
                material_remaining[m] = max(0.0, material_remaining.get(m, 0.0) - per_unit *
feasible_units)

        # Determine procurement plan if budget allows
        procurement = {}
        budget_left = self.budget
        # Target: fill 70% of unmet needs, adjusted by regime
        fill_ratio = 0.7 * clamp(knobs["inventory_buffer_factor"], 0.6, 1.4)

        for m, total_need in aggregate_needs.items():
            shortfall = max(0.0, total_need - available.get(m, 0.0))
            to_buy = shortfall * fill_ratio
            unit_cost = self.supplier_terms.get(m, {}).get("unit_cost", 0.0)
            cost = unit_cost * to_buy
            if cost <= budget_left and unit_cost > 0:
                procurement[m] = {"buy_qty": to_buy, "cost": cost}
                budget_left -= cost
            else:
                # Partial buy if budget constrained
                if unit_cost > 0 and budget_left > 0:
                    qty = budget_left / unit_cost
                    procurement[m] = {"buy_qty": qty, "cost": budget_left}
```

```
            budget_left = 0.0

        # KPIs
        planned_output_value = 0.0
        total_material_cost_implied = 0.0
        for sku, units in sku_allocation.items():
            # Optional: proxy unit margin to gauge value creation
            per_unit_material_cost = 0.0
            for m, per_unit in self.bom.get(sku, {}).items():
                per_cost = self.supplier_terms.get(m, {}).get("unit_cost", 0.0)
                per_unit_material_cost += per_unit * per_cost
            # Assume conservative margin proxy
            margin_proxy = max(0.0, 0.25 - 0.5 * safe_div(per_unit_material_cost, 100.0,
0.0))
            planned_output_value += units * margin_proxy
            total_material_cost_implied += units * per_unit_material_cost

        kpis = {
            "fill_rate_by_sku": {sku: safe_div(sku_allocation[sku], production_targets[sku],
0.0) for sku in sku_allocation},
            "budget_spent": self.budget - budget_left,
            "budget_remaining": budget_left,
            "planned_output_margin_proxy": planned_output_value,
            "material_cost_implied": total_material_cost_implied
        }

        return {
            "production_targets": production_targets,
            "sku_allocation_units": sku_allocation,
            "material_remaining": material_remaining,
            "procurement_plan": procurement,
            "policy_knobs": knobs,
            "kpis": kpis
        }

# ------------------------------
# Machine learning loop (policy refinement)
```

```
# ------------------------------

class AllocationLearner:
    """
    Lightweight policy learning loop (standard library):
    - Simulates allocation outcomes
    - Adjusts priority weights to improve margin proxy and fill rates
    """
    def __init__(self, forecaster, engine, initial_priority):
        self.forecaster = forecaster
        self.engine = engine
        self.priority = initial_priority.copy()  # {sku: weight}

    def step(self, regime, months_ahead=2):
        # Forecast and allocate
        forecast_by_sku = {}
        for sku in self.priority:
            f = self.forecaster.forecast(sku, months_ahead=months_ahead, regime=regime)
            forecast_by_sku[sku] = sum(f)  # horizon aggregate

        plan = self.engine.plan(forecast_by_sku, DualCircuitEconomics().signals(),
self.priority)

        # Reward: blend fill rates and margin proxy (you can tailor weights)
        fill_avg = moving_average(list(plan["kpis"]["fill_rate_by_sku"].values()),
len(plan["kpis"]["fill_rate_by_sku"]))
        margin = plan["kpis"]["planned_output_margin_proxy"]
        reward = 0.6 * fill_avg + 0.4 * clamp(margin, 0.0, 1000000.0) / 100000.0

        # Update priorities (policy gradient-like heuristic, no external libs)
        # Increase weight for SKUs with lower fill rate but high unit leverage (randomized
small steps to avoid cycles)
        for sku, fr in plan["kpis"]["fill_rate_by_sku"].items():
            step_dir = -0.04 if fr > 0.95 else 0.06
            noise = random.uniform(-0.01, 0.01)
            self.priority[sku] = clamp(self.priority[sku] + step_dir + noise, 0.05, 3.0)
```

```
        return plan, reward

# ------------------------------
# Interactive data collection (company context)
# ------------------------------

def collect_company_context():
    print("=== MANUFACTURING ALLOCATION INTELLIGENCE ===")
    company_name = input("Company name: ") or "Advanced Manufacturing Co."
    budget = float((input("Procurement budget ($): ") or "750000").replace(",", ""))

    # Materials
    print("\nEnter materials (comma separated), e.g., steel,aluminum,polymer")
    materials = [m.strip() for m in (input("Materials: ") or "steel,aluminum").split(",") if
m.strip()]

    on_hand = {}
    inbound = {}
    supplier_terms = {}
    for m in materials:
        on_hand[m] = float((input(f"{m} on-hand units: ") or "10000").replace(",", ""))
        inbound[m] = float((input(f"{m} inbound units (horizon): ") or "4000").replace(",",
""))
        unit_cost = float((input(f"{m} unit cost ($): ") or "2.5").replace(",", ""))
        lead_days = int((input(f"{m} lead time (days): ") or "21").replace(",", ""))
        supplier_terms[m] = {"unit_cost": unit_cost, "lead_time_days": lead_days}

    # SKUs and BOM
    print("\nEnter SKUs (comma separated), e.g., SKU_A,SKU_B")
    skus = [s.strip() for s in (input("SKUs: ") or "SKU_A,SKU_B").split(",") if s.strip()]

    bom = {}
    for sku in skus:
        print(f"\nDefine BOM for {sku}:")
        bom[sku] = {}
        for m in materials:
            per_unit = float((input(f"  {m} units per {sku}: ") or "2.0").replace(",", ""))
```

```
            bom[sku][m] = per_unit

    # Historical shipments (for forecasting)
    history_by_sku = {}
    for sku in skus:
        print(f"\nProvide last 12 months shipments for {sku} (comma numbers), e.g.,
120,130,...")
        raw = input("  History: ")
        if not raw.strip():
            # sensible defaults
            base = 100 + 10 * random.randint(0, 5)
            history_by_sku[sku] = [base + random.randint(-8, 12) for _ in range(12)]
        else:
            vals = []
            for token in raw.split(","):
                token = token.strip()
                if token:
                    try:
                        vals.append(float(token))
                    except:
                        pass
            history_by_sku[sku] = vals if vals else [100.0] * 12

    return {
        "company_name": company_name,
        "budget": budget,
        "materials": materials,
        "on_hand": on_hand,
        "inbound": inbound,
        "supplier_terms": supplier_terms,
        "skus": skus,
        "bom": bom,
        "history_by_sku": history_by_sku
    }

# ------------------------------
# Orchestrator (end-to-end run)
```

```
# ------------------------------

def main():
    ctx = collect_company_context()

    # Economics
    econ = DualCircuitEconomics()
    econ.fetch()
    regime = econ.regime
    signals = econ.signals()

    # Forecast + Allocation
    forecaster = DemandForecaster(ctx["history_by_sku"])
    engine = AllocationEngine(
        bom=ctx["bom"],
        on_hand=ctx["on_hand"],
        inbound=ctx["inbound"],
        budget=ctx["budget"],
        supplier_terms=ctx["supplier_terms"]
    )

    # Initial priorities: equal weights
    init_priority = {sku: 1.0 for sku in ctx["skus"]}
    learner = AllocationLearner(forecaster, engine, init_priority)

    # Run a few refinement steps (policy learning)
    best_plan = None
    best_reward = -1e9
    iterations = 5
    for _ in range(iterations):
        plan, reward = learner.step(regime=regime, months_ahead=2)
        if reward > best_reward:
            best_reward = reward
            best_plan = plan

    # Final plan on refined priorities
    forecast_by_sku = {}
```

```
    for sku in ctx["skus"]:
        forecast_by_sku[sku] = sum(forecaster.forecast(sku, months_ahead=2,
regime=regime))

    final_plan = engine.plan(forecast_by_sku, signals, learner.priority)

    # Report
    print("\n" + "="*72)
    print(f"ALLOCATION REPORT — {ctx['company_name']}")
    print("="*72)
    print(f"Regime: {regime}")
    print(f"FDR (proxy): {econ.fdr:.2f}")
    print(f"Signals: {[s['signal'] for s in signals]}")
    print("-"*72)

    print("\nProduction targets (units):")
    for sku, units in final_plan["production_targets"].items():
        print(f"  {sku}: {units:.0f}")

    print("\nAllocated production (units):")
    for sku, units in final_plan["sku_allocation_units"].items():
        fill = final_plan["kpis"]["fill_rate_by_sku"].get(sku, 0.0)
        print(f"  {sku}: {units:.0f}  (fill-rate {fill*100:.1f}%)")

    print("\nMaterial remaining after allocation:")
    for m, q in final_plan["material_remaining"].items():
        print(f"  {m}: {q:.0f}")

    print("\nProcurement plan:")
    if final_plan["procurement_plan"]:
        for m, p in final_plan["procurement_plan"].items():
            print(f"  {m}: buy {p['buy_qty']:.0f} units, cost ${p['cost']:,.0f}")
    else:
        print("  No purchases within budget constraints.")

    print("\nPolicy knobs (derived from regime signals):")
    for k, v in final_plan["policy_knobs"].items():
```

```
        print(f"  {k}: {v}")

    print("\nKPIs:")
    print(f"  Budget spent:       ${final_plan['kpis']['budget_spent']:,.0f}")
    print(f"  Budget remaining:    ${final_plan['kpis']['budget_remaining']:,.0f}")
    print(f"  Margin proxy (units):
{final_plan['kpis']['planned_output_margin_proxy']:.2f}")
    print(f"  Material cost implied: ${final_plan['kpis']['material_cost_implied']:,.0f}")

    print("\nRefined priorities by SKU:")
    for sku, w in learner.priority.items():
        print(f"  {sku}: {w:.2f}")

    print("\nRecommendation highlights:")
    if regime == "IMBALANCED_EXCESS":
        print("  - Reduce inventory exposure; tighten receivables; defer expansion CAPEX;
lock long-term fixed rates.")
    elif regime == "ASSET_LED_GROWTH":
        print("  - Slow inventory build; extend payables; secure and refinance while terms
are favorable.")
    elif regime == "REAL_ECONOMY_LEAD":
        print("  - Deploy strategically: counter-cyclical procurement/M&A; finance
suppliers; selective capacity expansion.")
    else:
        print("  - Optimize turnover; selective efficiency CAPEX; maintain balanced
buffers.")

    print("\nSaved snapshot: manufacturing_allocation_plan.txt")
    with open("manufacturing_allocation_plan.txt", "w") as f:
        f.write(json.dumps({
            "timestamp": datetime.now().isoformat(),
            "company_name": ctx["company_name"],
            "regime": regime,
            "fdr_proxy": econ.fdr,
            "signals": signals,
            "production_targets": final_plan["production_targets"],
            "sku_allocation_units": final_plan["sku_allocation_units"],
```

```
        "material_remaining": final_plan["material_remaining"],
        "procurement_plan": final_plan["procurement_plan"],
        "policy_knobs": final_plan["policy_knobs"],
        "kpis": final_plan["kpis"],
        "priorities": learner.priority
    }, indent=2))

if __name__ == "__main__":
    main()
```

Changes made

- Added dual-circuit economics module with an FDR proxy and regime detection to drive cycle-aware decisions across allocation, procurement, and buffers.
- Implemented a standard-library demand forecaster (exponential smoothing + seasonality + regime multiplier).
- Built a raw material allocation engine that honors BOMs, material constraints, budget, and supplier terms; integrates regime signals into inventory buffer policy, credit tightening, payables extension, and CAPEX gating.
- Added a lightweight learning loop (policy refinement) that adjusts SKU priorities to improve fill rates and margin proxy using a simple reward-driven heuristic.
- Created an interactive data collection flow to define materials, SKUs, BOMs, on-hand/inbound inventoies, supplier unit costs and lead times, budgets, and shipment histories.
- Produced an end-to-end orchestrator that:
    - Fetches economic context,
    - Forecasts demand,
    - Optimizes allocations,
    - Proposes procurement within budget,
    - Reports KPIs, policy knobs, and recommendations,
    - Saves a JSON snapshot to manufacturing_allocation_plan.txt.
- Ensured the entire system uses only Python’s standard library for easy copy/paste execution in a basic simulator, consistent with your philosophy and market beliefs about asset-led regimes and counter-cyclical manufacturing strategy.

```
#!/usr/bin/env python3

# -*- coding: utf-8 -*-


"""

MANUFACTURING ALLOCATION INTELLIGENCE SYSTEM (SAAS CORE)

Cycle-aware raw material allocation, demand forecasting, and policy signals

Aligned with dual-circuit economics (FDR) and counter-cyclical principles

Standard library only: ready to copy/paste and run.

"""


import math

import json

import random

from datetime import datetime

from urllib.request import urlopen


# ------------------------------

# Utility functions

# ------------------------------
```

```
def clamp(x, lo, hi): return max(lo, min(hi, x))
def safe_div(a, b, default=0.0): return a / b if b else default
def moving_average(series, window):
    if window <= 0 or not series: return 0.0
    window = min(window, len(series))
    return sum(series[-window:]) / window
def exponential_smoothing(series, alpha=0.4):
    if not series: return 0.0
    s = series[0]
    for x in series[1:]:
        s = alpha * x + (1 - alpha) * s
    return s


# ------------------------------
# Dual-circuit economics (FDR)
# ------------------------------


class DualCircuitEconomics:
    def __init__(self):
        self.data = {}
        self.fdr = 1.0
        self.regime = "HEALTHY_EXPANSION"
```

```
    def fetch(self):
        try:
            with urlopen('https://api.factoryofthefuture.ai/economic-indicators') as
response:
                data = json.loads(response.read().decode())
        except Exception:
            data = {
                "manufacturing_pmi": 52.5,
                "core_pce": 0.032,
                "ci_loans_growth": 0.03,
                "margin_debt_growth": 0.08,
                "asset_price_index": 1.12,
                "last_updated": datetime.now().isoformat(),
                "source": "Dual-Circuit Model (fallback)"
            }
        self.data = data
        self._compute_fdr()
        self._detect_regime()
        return self.data


    def _compute_fdr(self):
        asset_growth = self.data.get("margin_debt_growth", 0.06)
```

```
    asset_index  = self.data.get("asset_price_index", 1.08)
    ci_growth    = self.data.get("ci_loans_growth", 0.02)
    core_pce     = self.data.get("core_pce", 0.03)
    denom = ci_growth * core_pce if ci_growth * core_pce > 0 else 0.0001
    self.fdr = clamp((asset_growth * asset_index) / denom, 0.2, 5.0)

def _detect_regime(self):
    if self.fdr > 2.0: self.regime = "IMBALANCED_EXCESS"
    elif self.fdr > 1.5: self.regime = "ASSET_LED_GROWTH"
    elif self.fdr < 0.8: self.regime = "REAL_ECONOMY_LEAD"
    else: self.regime = "HEALTHY_EXPANSION"

def signals(self):
    if self.regime == "IMBALANCED_EXCESS":
        return [{"signal":"REDUCE_INVENTORY","intensity":0.8},
                {"signal":"TIGHTEN_CREDIT_TERMS","intensity":0.7},
                {"signal":"DEFER_EXPANSION_CAPEX","intensity":0.9},
                {"signal":"REFINANCE_LONG_FIXED","intensity":0.9}]
    elif self.regime == "ASSET_LED_GROWTH":
        return [{"signal":"SLOW_INVENTORY_BUILD","intensity":0.6},
                {"signal":"EXTEND_PAYABLES","intensity":0.6},
                {"signal":"LOCK_RATES","intensity":0.7}]
```

```
        elif self.regime == "REAL_ECONOMY_LEAD":
            return [{"signal":"COUNTER_CYCLICAL_BUY","intensity":0.9},
                    {"signal":"EXPAND_CAPACITY_SELECTIVE","intensity":0.7},
                    {"signal":"SUPPLY_CHAIN_FINANCE","intensity":0.8}]
        else:
            return [{"signal":"OPTIMIZE_TURNOVER","intensity":0.5},
                    {"signal":"SELECTIVE_CAPEX","intensity":0.5}]


# ------------------------------
# Demand forecaster
# ------------------------------


class DemandForecaster:
    def __init__(self, history_by_sku): self.history_by_sku = history_by_sku or {}
    def forecast(self, sku, months_ahead=3, regime="HEALTHY_EXPANSION"):
        history = self.history_by_sku.get(sku, [])
        base = exponential_smoothing(history, alpha=0.45)
        regime_factor =
{"IMBALANCED_EXCESS":0.92,"ASSET_LED_GROWTH":0.96,

"HEALTHY_EXPANSION":1.00,"REAL_ECONOMY_LEAD":1.05}.get(regime
,1.0)
        seasonality = moving_average(history, 12)
```

```
        season_factor = clamp(base/seasonality,0.85,1.15) if seasonality>0 else 1.0
        forecasts = []
        level = base * regime_factor * season_factor
        for h in range(1, months_ahead+1):
            damp = 1.0 - 0.06*(h-1)
            forecasts.append(max(0.0, level*damp))
        return forecasts


# ------------------------------
# Allocation engine
# ------------------------------


class AllocationEngine:
    def __init__(self,bom,on_hand,inbound,budget,supplier_terms):
        self.bom=bom; self.on_hand=on_hand; self.inbound=inbound
        self.budget=budget; self.supplier_terms=supplier_terms
    def _material_need_for_sku(self,sku,units):
        need={}
        for m,per_unit in self.bom.get(sku,{}).items():
            need[m]=need.get(m,0.0)+per_unit*units
        return need
    def _available_material(self,m): return
self.on_hand.get(m,0.0)+self.inbound.get(m,0.0)
```

```
    def plan(self,forecast_by_sku,regime_signals,priority_by_sku):

knobs={"inventory_buffer_factor":1.0,"tighten_credit":False,"extend_payables":F
alse,"capex_gate":False}

        for s in regime_signals:

            if s["signal"] in
("REDUCE_INVENTORY","SLOW_INVENTORY_BUILD"):

                knobs["inventory_buffer_factor"]*=(1.0-0.15*s["intensity"])

            if s["signal"]=="TIGHTEN_CREDIT_TERMS":
knobs["tighten_credit"]=True

            if s["signal"]=="EXTEND_PAYABLES": knobs["extend_payables"]=True

            if s["signal"]=="DEFER_EXPANSION_CAPEX":
knobs["capex_gate"]=True

        total_w=sum(priority_by_sku.values()) or 1.0

        norm_priority={sku:w/total_w for sku,w in priority_by_sku.items()}


production_targets={sku:max(0.0,units*(1.0+0.10*knobs["inventory_buffer_facto
r"]))

                        for sku,units in forecast_by_sku.items()}

        aggregate_needs={}

        for sku,units in production_targets.items():

            need=self._material_need_for_sku(sku,units)

            for m,q in need.items():
aggregate_needs[m]=aggregate_needs.get(m,0.0)+q

        available={m:self._available_material(m) for m in aggregate_needs}
```

```
        sku_allocation={sku:0.0 for sku in production_targets}

        material_remaining=available.copy()

        for sku,units in sorted(production_targets.items(),key=lambda
kv:-norm_priority.get(kv[0],0)):

            caps=[material_remaining.get(m,0.0)/per_unit for m,per_unit in
self.bom.get(sku,{}).items() if per_unit>0]

            feasible=min([units]+caps) if caps else 0.0

            sku_allocation[sku]=feasible

            for m,per_unit in self.bom.get(sku,{}).items():


material_remaining[m]=max(0.0,material_remaining.get(m,0.0)-per_unit*feasible
)

        procurement={}; budget_left=self.budget

        fill_ratio=0.7*clamp(knobs["inventory_buffer_factor"],0.6,1.4)

        for m,total_need in aggregate_needs.items():

            shortfall=max(0.0,total_need-available.get(m,0.0))

            to_buy=shortfall*fill_ratio

            unit_cost=self.supplier_terms.get(m,{}).get("unit_cost",0.0)

            cost=unit_cost*to_buy

            if unit_cost>0:

                if cost<=budget_left:

                    procurement[m]={"buy_qty":to_buy,"cost":cost}; budget_left-=cost

                elif budget_left>0:

                    qty=budget_left/unit_cost
```

procurement[m]